CHAPTER 2

UNIT – II

Solving Problems by Searching: Problem – Solving Agents
Uninformed Search Strategies: Best-First Search, Breadth-First Search, Uniform-Cost Search, Depth-First Search, Iterative Deepening Search and Bidirectional Search
Informed Search Strategies: Greedy Best-First Search, A* Search

🧠 SOLVING PROBLEMS BY SEARCHING
(Problem-Solving Agents in AI)

📌 1. WHAT IS A PROBLEM-SOLVING AGENT?

A **Problem-Solving Agent** is an intelligent agent that:

- **Decides what to do by searching** for a sequence of actions
- Aims to reach a **goal state** from an **initial state**

👉 It works when:

- The environment is **fully observable**
- The agent has a **goal**
- The agent can **model the environment**

📌 2. BASIC IDEA (INTUITION)

Instead of reacting immediately, the agent:

1. **Thinks (searches)** possible actions
2. **Finds a path**
3. **Executes the solution**

👉 Example:

- You want to go from home → college
- You explore routes → choose best path → travel

📌 3. COMPONENTS OF A PROBLEM

A problem in AI is defined by **5 components**:

1. Initial State

- Starting point
  👉 Example: Arshad at home

2. Actions

- Possible moves from a state
  👉 Example: Walk, take bus, bike

3. Transition Model

- Result of an action
  👉 Example: "Go by bus → reach stop"

4. Goal Test

- Checks if goal is reached
  👉 Example: Reached college?

5. Path Cost

- Cost of the path (time, distance, money)
  👉 Example: Shortest time route

📌 4. STATE SPACE

🔷 Definition:

All possible states reachable from the initial state

👉 Represented as a **graph/tree**

- Nodes = States
- Edges = Actions

📌 5. SEARCH PROCESS

The agent explores the state space to find a solution.

🔁 Steps:

1. Formulate goal
2. Formulate problem
3. Search for solution
4. Execute actions

📌 6. TYPES OF SEARCH STRATEGIES

🔷 1. Uninformed Search (Blind Search)

No extra knowledge, only problem definition used.

Algorithms:

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Uniform Cost Search (UCS)

🔷 2. Informed Search (Heuristic Search)

Uses extra knowledge (**heuristic function h(n)**)

Algorithms:

- Greedy Best First Search
- A* Search

📌 7. TREE SEARCH vs GRAPH SEARCH

Feature	Tree Search	Graph Search
Repeated States	Allowed	Avoided
Memory	Less	More
Efficiency	Low	High

📌 8. PROPERTIES OF SEARCH ALGORITHMS

🔷 1. Completeness

- Will it find a solution?

🔷 2. Optimality

- Will it find best solution?

🔷 3. Time Complexity

- How fast?

🔷 4. Space Complexity

- Memory usage

📌 9. EXAMPLE PROBLEM

🔷 8-Puzzle Problem

Components:

- Initial State → scrambled tiles
- Goal State → ordered tiles
- Actions → move blank tile
- Cost → number of moves

📌 10. SIMPLE PROBLEM-SOLVING AGENT (ALGORITHM)

```
1. Update current state
2. If no solution:
      – Formulate goal
      – Formulate problem
      – Search for solution
3. Execute first action of solution
4. Repeat
```

📌 11. IMPORTANT TERMS (EXAM QUICK REVISION)

- **State** → Situation at a point
- **Node** → State + parent + cost
- **Path** → Sequence of actions
- **Solution** → Path to goal
- **Search Tree** → Exploration structure

📌 12. REAL-LIFE APPLICATIONS

- GPS navigation
- Robot path planning
- Game playing (Chess, etc.)
- Puzzle solving

📌 13. ADVANTAGES

✔️ Systematic approach
✔️ Finds optimal solution (in some algorithms)
✔️ Widely applicable

📌 14. LIMITATIONS

❌ High memory usage
❌ Time-consuming for large problems
❌ Needs well-defined problem

📌 15. EXAM ANSWER STRUCTURE (VERY IMPORTANT)

If question comes:

✍️ Write in this order:

1. Definition
2. Components (5 points)
3. State space explanation
4. Search process
5. Types of search
6. Example
7. Conclusion

🔥 FINAL REVISION SUMMARY

👉 Problem-solving agent = **Search + Goal + Actions**
👉 Core idea = **Find path from initial → goal**
👉 Key elements = **State, Actions, Goal, Cost**
👉 Two types = **Uninformed & Informed search**

🧠 UNINFORMED SEARCH STRATEGIES

(Breadth-First Search & Best-First Search)

📌 1. INTRODUCTION TO UNINFORMED SEARCH

🔷 Definition:

Uninformed Search (Blind Search) means:

- The agent has **no extra knowledge (heuristics)**
- Uses only:
  - Initial state
  - Actions
  - Goal test
  - Path cost

👉 It explores the state space **systematically**

📌 2. BREADTH-FIRST SEARCH (BFS)

🔷 Definition:

BFS explores the **shallowest nodes first** (level by level)

👉 Uses **Queue (FIFO)**

🔷 Working Principle

1. Start from initial state
2. Expand all nodes at current depth
3. Move to next level
4. Repeat until goal found

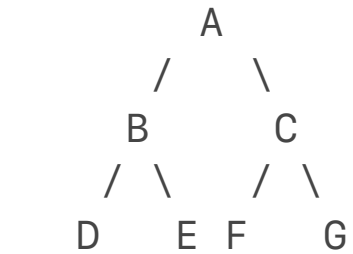
🔷 Data Structure Used

- **Queue (FIFO)**

🔷 Algorithm (Exam Ready)

```
1. Initialize queue with initial state
2. Repeat:
   a. Remove front node from queue
   b. If goal → return solution
   c. Expand node
   d. Add all children to queue (end)
```

🔷 Example (Concept)



👉 BFS order:

A → B → C → D → E → F → G

🔷 Properties

Property	Value
Completeness	✅ Yes
Optimality	✅ Yes (if cost is same)
Time Complexity	O(b^d)
Space Complexity	O(b^d)

👉 b = branching factor, d = depth

🔷 Advantages

✔️ Finds shortest path
✔️ Simple to implement

🔷 Disadvantages

❌ High memory usage
❌ Slow for deep trees

📌 3. BEST-FIRST SEARCH (IMPORTANT CLARITY)

⚠️ **Important**:
Best-First Search is often considered **Informed Search**, but sometimes included here for comparison.

🔷 Definition:

Selects node based on **best value of evaluation function f(n)**
👉 Expands the most promising node first

🔷 Types of Best-First Search

1. Greedy Best-First Search

- Uses **f(n) = h(n)**
- Chooses node closest to goal

2. A* Search

- Uses **f(n) = g(n) + h(n)**
- Combines cost + heuristic

🔷 Data Structure Used

- **Priority Queue**

🔷 Algorithm (General Form)

```
1. Initialize priority queue with initial node
2. Repeat:
   a. Remove best node (lowest f(n))
   b. If goal → return solution
   c. Expand node
   d. Add children with priority
```

🔷 Example Idea

👉 If goal is closest city:

- Greedy → chooses nearest estimated distance
- A* → considers distance travelled + estimated

🔷 Properties (General)

Property	Value
Completeness	❌ Not always
Optimality	❌ Not always (except A*)
Time Complexity	Depends
Space Complexity	High

🔷 Advantages

✔️ Faster with good heuristic
✔️ Focused search

🔷 Disadvantages

❌ Needs heuristic
❌ Can give wrong solution

📌 4. BFS vs BEST-FIRST SEARCH

Feature	BFS	Best-First
Knowledge	No heuristic	Uses heuristic
Strategy	Level-wise	Best node first
Data Structure	Queue	Priority Queue
Optimal	Yes	Depends
Speed	Slow	Faster (if good heuristic)

📌 5. KEY DIFFERENCE (VERY IMPORTANT)

👉 **BFS**

- Blind search
- Explores all equally

👉 **Best-First**

- Intelligent search
- Focuses on promising paths

📌 6. EXAM TIPS (VERY IMPORTANT)

If question is:

✍️ "Explain BFS"

Write:

1. Definition
2. Algorithm
3. Diagram/example
4. Properties
5. Advantages & disadvantages

✍️ "Differentiate BFS and Best-First"

- Make table (very scoring)

🔥 FINAL REVISION SUMMARY

👉 BFS = **Level order + Queue + Shortest path**
👉 Best-First = **Priority-based + Heuristic**
👉 BFS = **Complete & Optimal**
👉 Best-First = **Fast but not always correct**

You're not wrong to feel that way.
What you got earlier was **exam-summary level**, not **concept-master level**.
Now I'll give you **clear + deeper + structured notes** so that:

- You can **understand from scratch**
- AND still **write in exam properly**

🧠 UNIFORM COST SEARCH (UCS) — DEEP + CLEAR NOTES

📌 1. INTUITION (VERY IMPORTANT)

👉 Think like this:
You are travelling and:

- Different roads have **different costs (time/money)**
- You want the **cheapest route**, NOT shortest steps

✔ That's exactly what UCS does

👉 It always chooses the **least total cost path so far**

📌 2. CORE IDEA

- Expand node with **minimum path cost g(n)**
- It does **NOT care about depth**
- It does **NOT use heuristic**

📌 3. HOW UCS WORKS (STEP-BY-STEP FLOW)

1. Start from initial node (cost = 0)
2. Put it in **priority queue**
3. Repeat:
    - Pick node with **lowest cost**
    - Check goal
    - Expand node
    - Add children with updated cost

📌 4. WHY PRIORITY QUEUE?

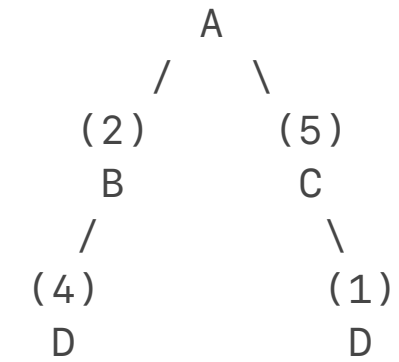
👉 Because we always need:

- "Give me the **lowest cost node** quickly"

✔️ Priority Queue ensures:

- Smallest cost → processed first

📌 5. WORKING EXAMPLE (VERY IMPORTANT)



Paths:

- A → B → D = 2 + 4 = 6
- A → C → D = 5 + 1 = 6

👉 UCS explores based on **cost order**, not level

📌 6. KEY POINT (VERY IMPORTANT)

👉 UCS is basically:

🔥 **"BFS with costs"**

✔️ If all costs = 1

➡️ UCS behaves exactly like BFS

📌 7. PROPERTIES (UNDERSTAND, NOT MEMORIZE)

✅ Completeness

✔️ Yes, if cost > 0

👉 Because it will eventually explore all cheaper paths

✅ Optimality

✔️ YES (Very important)

👉 First time goal is reached = **minimum cost path**

⏱️ Time Complexity

- Depends on cost range
- Can be large

💾 Space Complexity

- High (stores all nodes)

📌 8. WHERE UCS FAILS?

❌ If:

- Costs are very small → many expansions
- State space is huge → memory issue

📌 9. REAL-LIFE EXAMPLE

- Google Maps shortest path
- Network routing
- Cheapest flight route

📌 10. COMMON CONFUSION

❌ BFS = shortest path

✔️ Only when cost is same

❌ UCS = shortest path

✔️ Actually = **least cost path**

🧠 DEPTH-FIRST SEARCH (DFS) — DEEP + CLEAR NOTES

📌 1. INTUITION

👉 Imagine exploring a maze:

- You go **as deep as possible**
- If stuck → come back → try another path

✔️ That's DFS

📌 2. CORE IDEA

- Expand **deepest unexplored node**
- Follow one path **completely**
- Then **backtrack**

📌 3. HOW DFS WORKS

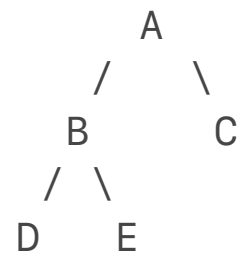
1. Start at initial node
2. Push into stack
3. Repeat:
    - Pop node
    - Check goal
    - Expand node
    - Push children

📌 4. WHY STACK?

👉 Because:

- Last inserted node is explored first
  ✔️ This creates **depth behavior**

📌 5. WORKING EXAMPLE



DFS Traversal:

A → B → D → (backtrack) → E → (backtrack) → C

📌 6. KEY CHARACTERISTIC

👉 DFS:

- Goes **deep first**
- Doesn't care about:
    - Cost
    - Shortest path

📌 7. PROPERTIES (UNDERSTAND CLEARLY)

❌ Completeness

- NOT guaranteed
  👉 Can get stuck in infinite depth

❌ Optimality

- NOT optimal
  👉 First solution may not be best

⏱️ Time Complexity

- O(b^m)

👉 b = branching factor
👉 m = maximum depth

💾 Space Complexity

- O(bm) → **very low**

✔ Biggest advantage of DFS

📌 8. WHERE DFS IS USEFUL?

✔ When:

- Memory is limited
- Solution is very deep
- Don't care about optimal path

📌 9. WHERE DFS FAILS?

❌ Infinite loops
❌ Finds wrong (non-optimal) solution
❌ Can miss shallow solution

📌 10. REAL-LIFE EXAMPLE

- Puzzle solving
- Maze exploration
- Backtracking problems

⚔️ UCS vs DFS — DEEP UNDERSTANDING

Concept	UCS	DFS
Goal	Minimum cost	Go deep
Strategy	Smart (cost-based)	Blind (depth-based)
Data Structure	Priority Queue	Stack
Optimal	✅ Yes	❌ No
Complete	✅ Yes	❌ No
Memory	❌ High	✅ Low

🔥 FINAL UNDERSTANDING (VERY IMPORTANT)

👉 UCS =

“Careful thinker”

✔️ Finds best path

❌ Takes time + memory

👉 DFS =

“Risk taker”

✔️ Fast & low memory

❌ Can go wrong

🎯 HOW TO REMEMBER (SUPER SIMPLE)

- BFS → Level
- DFS → Depth
- UCS → Cost

🧠 ITERATIVE DEEPENING SEARCH (IDS)

📌 1. INTUITION (MOST IMPORTANT)

👉 Problem with searches:

- **BFS** → finds shortest path but uses **huge memory**
- **DFS** → uses less memory but can go **wrong/infinite**

👉 IDS = **combination of BFS + DFS**

✔️ It tries:

- DFS with depth limit = 0
- Then DFS with depth limit = 1
- Then DFS with depth limit = 2
- … until goal is found

📌 2. CORE IDEA

👉 “Search deeper step by step”

Instead of going infinitely deep:

- It **limits depth**
- Gradually increases limit

📌 3. HOW IDS WORKS (STEP-BY-STEP)

1. Set depth limit = 0
2. Perform **Depth-Limited DFS**
3. If goal not found:
    - Increase limit (limit = limit + 1)
4. Repeat until goal is found

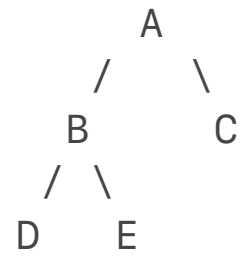
📌 4. WHAT IS DEPTH-LIMITED SEARCH?

👉 DFS but with a **maximum depth allowed**

✔️ It prevents:

- Infinite loops
- Going too deep

📌 5. WORKING EXAMPLE



IDS Steps:

- Depth 0 → A
- Depth 1 → A, B, C
- Depth 2 → A, B, D, E, C

👉 Repeats nodes again, but controlled

📌 6. IMPORTANT OBSERVATION

❗ Yes, IDS **repeats work**

BUT:

✔️ Most nodes are at deeper levels

✔️ So repetition cost is small

📌 7. PROPERTIES

✅ Completeness

✔️ Yes (like BFS)

✅ Optimality

✔️ Yes (if cost is same)

⏱️ Time Complexity

- O(b^d)

👉 Same as BFS (approx)

💾 Space Complexity

- O(bd)

👉 Much less than BFS

📌 8. WHY IDS IS POWERFUL?

👉 Combines:

- BFS's correctness
- DFS's low memory

✔️ Best for:

- Large search space
- Unknown depth

📌 9. REAL-LIFE IDEA

- Searching in games (chess engines use variations)
- When you don't know how deep solution is

📌 10. LIMITATIONS

❌ Repeats nodes
❌ Slightly slower than BFS

🧠 BIDIRECTIONAL SEARCH

📌 1. INTUITION (VERY IMPORTANT)

👉 Instead of searching:

- Only from **start → goal**

👉 We search:

- From **start → forward**
- From **goal → backward**

✔️ And meet in the middle

📌 2. CORE IDEA

👉 "Meet in the middle"

This reduces:

- Search depth
- Time complexity

📌 3. HOW IT WORKS

1. Start two searches:
    - Forward search from initial state
    - Backward search from goal
2. Expand both sides
3. Stop when:
   👉 Both searches **intersect (common node found)**

📌 4. VISUAL IDEA

```
Start → → → ← ← ← Goal
          Meet
```

👉 Instead of depth = d

Each side searches ≈ d/2

📌 5. DATA STRUCTURE

- Usually uses:
  - BFS (both sides)
  - Queue

📌 6. WHY IT IS FAST?

👉 Normal BFS:

- Time = O(b^d)

👉 Bidirectional:

- Time = O(b^(d/2))

🔥 HUGE IMPROVEMENT

📌 7. PROPERTIES

✅ Completeness

✔️ Yes (if BFS used)

✅ Optimality

✔️ Yes (if costs equal)

⏱️ Time Complexity

- O(b^(d/2)) → very fast

💾 Space Complexity

- High (stores two frontiers)

📌 8. CONDITIONS TO USE

👉 Must satisfy:

1. Goal state must be known
2. Reverse actions must be possible
3. State comparison should be easy

📌 9. WHERE IT FAILS?

❌ If:

- Goal is unknown
- Reverse moves not possible

- Huge memory needed

📌 10. REAL-LIFE EXAMPLE

- GPS navigation
- Puzzle solving (like 8-puzzle)

⚔️ IDS vs BIDIRECTIONAL SEARCH

Feature	IDS	Bidirectional
Strategy	Increase depth gradually	Search from both sides
Memory	Low	High
Speed	Moderate	Very fast
Optimal	Yes	Yes
Use case	Unknown depth	Known goal

🔥 FINAL UNDERSTANDING

👉 IDS =
"Try depth step by step"
✔️ Safe + memory efficient
👉 Bidirectional =
"Attack from both sides"
✔️ Very fast

🎯 SUPER SHORT REVISION

- IDS = DFS + depth limit + repetition
- Bidirectional = forward + backward search
- IDS = saves memory
- Bidirectional = saves time

🧠 INFORMED SEARCH STRATEGIES
(Greedy Best-First Search & A* Search)

📌 1. WHAT IS INFORMED SEARCH?

🔷 Definition:

Informed Search (Heuristic Search) uses **extra knowledge** to guide the search towards the goal.

👉 Unlike uninformed search:

- It does **not explore blindly**
- It uses a **heuristic function h(n)**

📌 2. WHAT IS HEURISTIC FUNCTION?

🔷 Definition:

A **heuristic function h(n)** estimates:

👉 "How far is the current node from the goal?"

🔷 Example:

- Straight-line distance in maps
- Tiles out of place in 8-puzzle

🔷 Important Types

✔️ Admissible Heuristic

- Never overestimates cost
  👉 h(n) ≤ actual cost

✔️ Consistent Heuristic

- Follows triangle inequality
  👉 h(n) ≤ cost(n→n') + h(n')

🧠 GREEDY BEST-FIRST SEARCH (GBFS)

📌 3. INTUITION

👉 Imagine:

You always choose the path that **looks closest to goal**

✔️ You don't care about cost already travelled

✔️ Only care about **estimated distance**

📌 4. CORE IDEA

👉 Select node with:

🔥 f(n) = h(n)

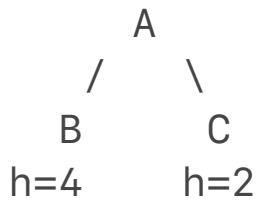
✔️ Only heuristic matters

📌 5. HOW IT WORKS

1. Start from initial node
2. Put in **priority queue**
3. Repeat:

- Select node with **lowest h(n)**
- Check goal
- Expand node
- Add children

📌 6. WORKING EXAMPLE



👉 Greedy chooses:

C (because h is smaller)

📌 7. CHARACTERISTICS

✔️ Fast

✔️ Focused search

❌ Can ignore better paths

📌 8. PROPERTIES

Property	Value
Completeness	❌ No
Optimality	❌ No
Time Complexity	O(b^m)
Space Complexity	High

📌 9. WHERE IT FAILS?

👉 Example:

- Path looks close but actually longer

❌ Gets stuck in:

- Local minima
- Wrong direction

📌 10. REAL-LIFE IDEA

- Fast GPS suggestion (not always best)
- Game AI quick decisions

🧠 A* SEARCH (MOST IMPORTANT 🔥)

📌 11. INTUITION

👉 A* is **smart combination of:**

- UCS (actual cost)
- Greedy (heuristic)

✔️ It balances:

- Distance travelled
- Distance remaining

📌 12. CORE FORMULA

🔥 f(n) = g(n) + h(n)

Where:

- g(n) = cost from start to node
- h(n) = estimated cost to goal

📌 13. MEANING

👉 A* says:

"Choose node with **least total estimated cost**"

📌 14. HOW A* WORKS

1. Start with initial node
2. Put in priority queue
3. Repeat:
    - Select node with lowest f(n)
    - Check goal
    - Expand node
    - Update costs

📌 15. WORKING EXAMPLE

```
Path 1: g=3, h=5 → f=8
Path 2: g=6, h=1 → f=7
```

👉 A* chooses:

✔️ Path 2 (because f is smaller)

📌 16. WHY A* IS BEST?

👉 Because:

- Considers **past + future**
- Avoids greedy mistakes
- Avoids UCS slowness

📌 17. PROPERTIES (VERY IMPORTANT)

✅ Completeness

✔️ Yes

✅ Optimality
✔️ Yes (if heuristic is admissible)

⏱️ Time Complexity
- Exponential (worst case)

💾 Space Complexity
- High

📌 18. KEY CONDITION
👉 For A* to be optimal:
✔️ h(n) must be admissible

📌 19. SPECIAL CASES (VERY IMPORTANT)
👉 If:
- h(n) = 0 → A* becomes UCS

👉 If:
- g(n) ignored → becomes Greedy

📌 20. REAL-LIFE APPLICATIONS
- Google Maps
- Robot navigation
- Game AI
- Pathfinding

⚔️ GREEDY vs A* (VERY IMPORTANT TABLE)

Feature	Greedy	A*
Formula	f = h(n)	f = g(n) + h(n)
Focus	Goal only	Cost + Goal
Optimal	❌ No	✅ Yes
Completeness	❌ No	✅ Yes
Speed	Fast	Moderate
Accuracy	Low	High

📌 21. DEEP UNDERSTANDING

👉 Greedy =
“Looks good now”
❌ Can be wrong
👉 A* =
“Looks good + logically correct”
✔️ Best choice

🔥 FINAL REVISION

👉 Heuristic = estimate to goal
👉 Greedy = h(n) only
👉 A* = g(n) + h(n)
👉 A* = most important algorithm